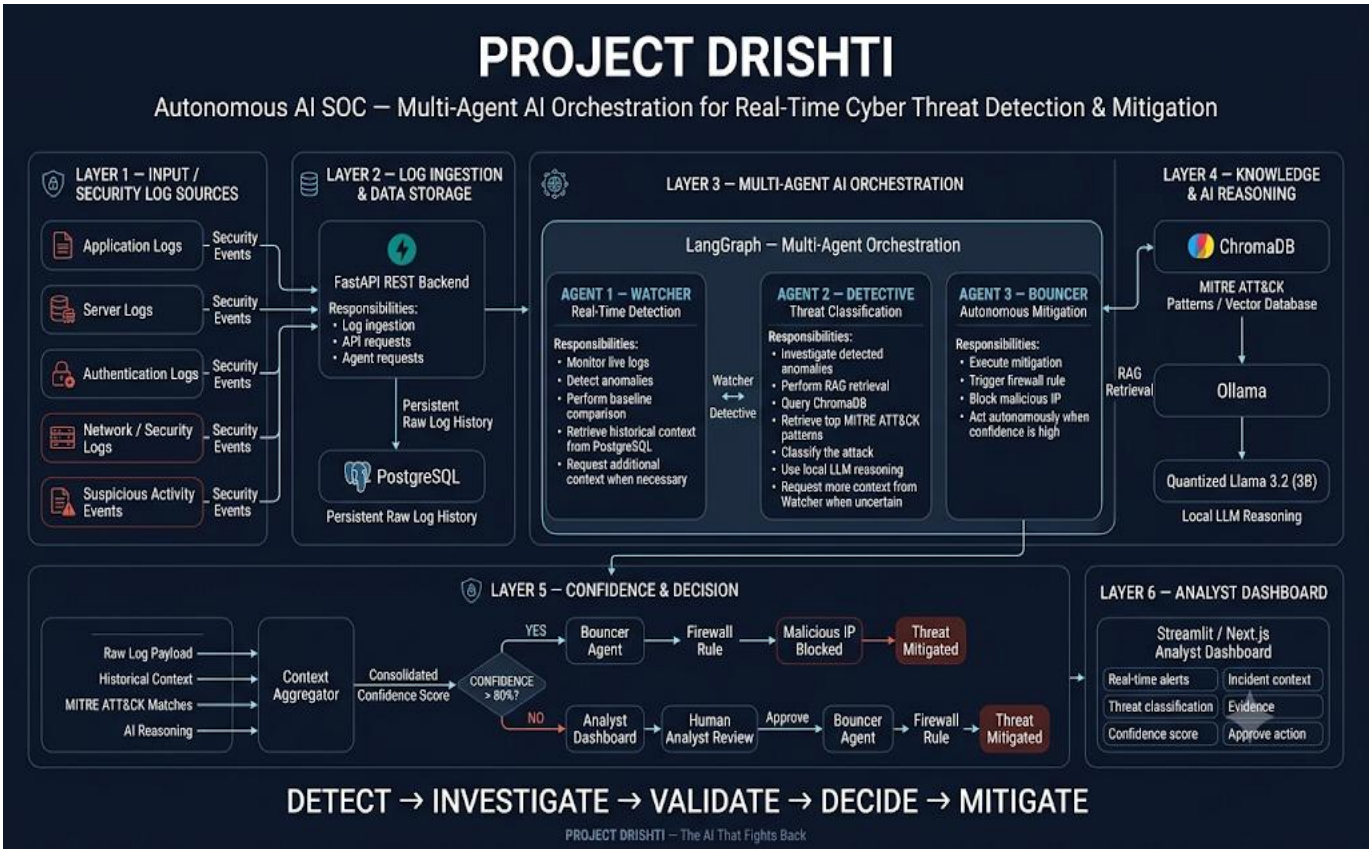
# Project Drishti: Core Architecture & Workflow Design

## Title: Autonomous AI SOC — Multi-Agent AI Orchestration

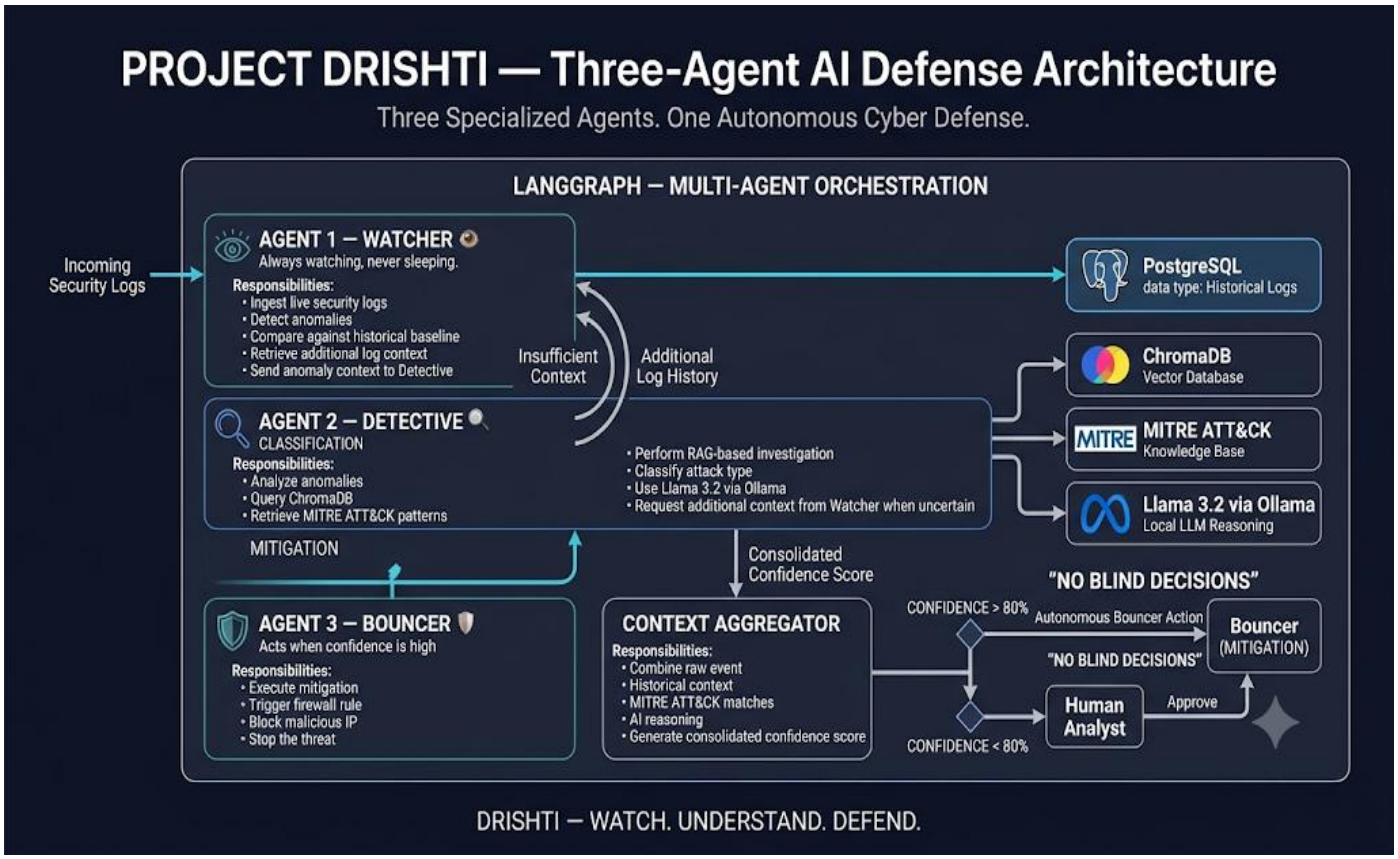
**Description:** This diagram provides a comprehensive, end-to-end view of Project Drishti's layered architecture. It illustrates the complete lifecycle of a security event, starting from raw log ingestion via the FastAPI backend (Layer 1 & 2), moving into the LangGraph-powered multi-agent orchestration engine (Layer 3), and leveraging local AI reasoning (Layer 4). It highlights the critical decision-making gateway (Layer 5) where the system aggregates context to determine whether to act autonomously or escalate the threat to a human analyst via the dashboard.



**Title:** Three Specialized Agents. One Autonomous Cyber Defense.

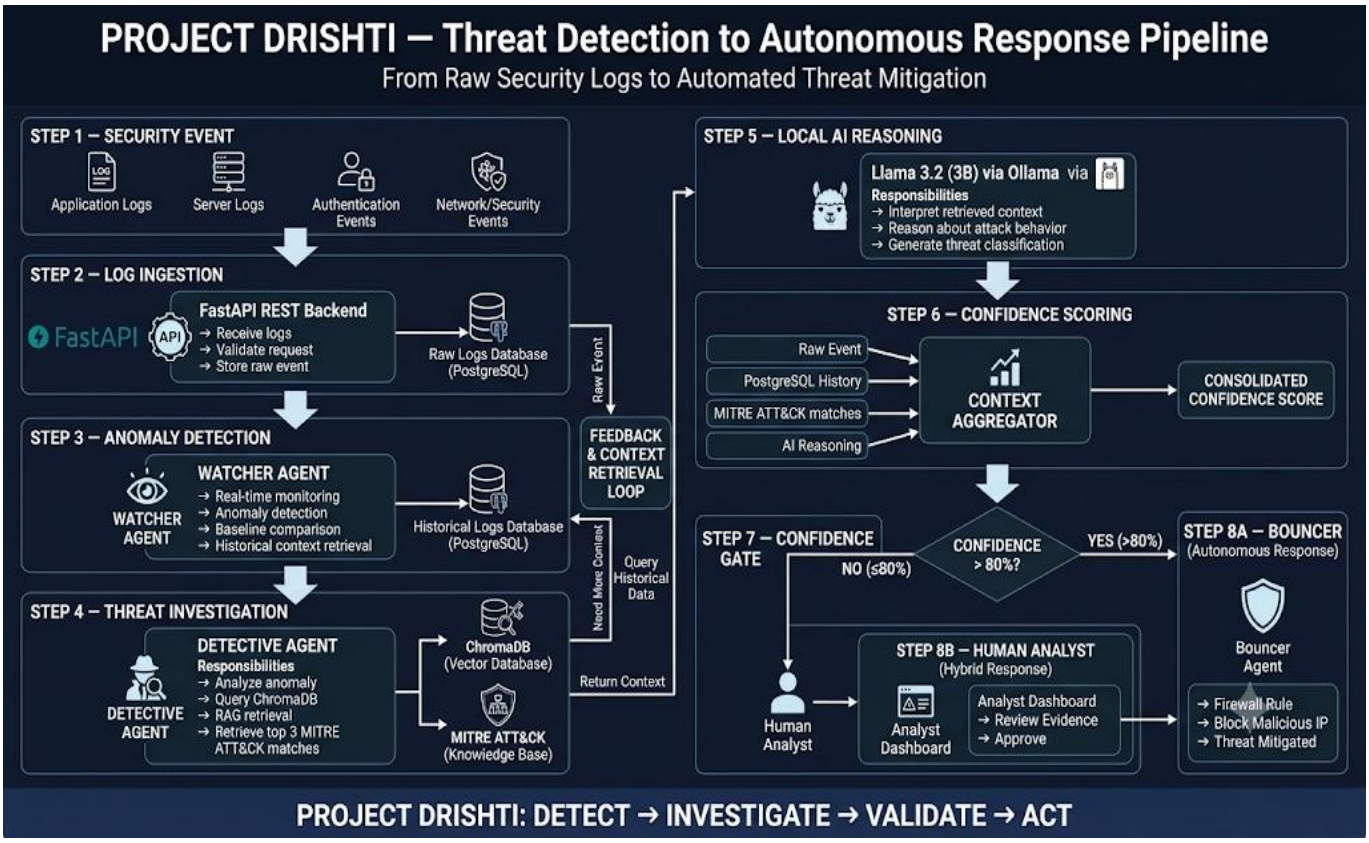
**Description:** This diagram zooms in on the core intelligence of Project Drishti: the LangGraph multi-agent orchestration layer. It details the specific responsibilities of the three specialized agents:

- **Watcher (Detection):** Constantly monitors live logs and detects anomalies against historical baselines.
- **Detective (Classification):** Investigates anomalies using RAG (Retrieval-Augmented Generation), querying ChromaDB for MITRE ATT&CK patterns, and reasoning via local Llama 3.2 models.
- **Bouncer (Mitigation):** Executes defensive actions, such as triggering firewall rules, when threat confidence is high. It also maps the internal feedback loops, demonstrating how agents can request more context from one another to ensure "No Blind Decisions."



# Title: From Raw Security Logs to Automated Threat Mitigation

**Description:** This diagram outlines the sequential, 8-step chronological workflow of a security incident within Project Drishti. It maps the journey of a data packet from the initial suspicious event (Step 1) through log ingestion (Step 2), anomaly detection (Step 3), and AI-driven investigation (Step 4 & 5). The pipeline emphasizes the **Confidence Gate (Step 7)**, which serves as the system's safety mechanism. If the consolidated confidence score exceeds 80%, the system takes immediate autonomous action (Step 8A); otherwise, it gracefully degrades to a human-in-the-loop hybrid response (Step 8B).



# Title: On-Premise Multi-Agent AI Cyber Defense Stack

**Description**: This diagram details the specific technology stack and deployment strategy for Project Drishti, emphasizing data privacy and operational independence. It maps the software components to their architectural roles, featuring FastAPI for backend routing, PostgreSQL for persistent relational log storage, and ChromaDB for vector-based MITRE ATT&CK knowledge retrieval. Crucially, it highlights the On-Premise / Self-Hosted Environment, showcasing that the LangGraph orchestration and Ollama-quantized Llama 3.2 (3B) inference run entirely locally, ensuring strict data privacy with zero reliance on external cloud LLM APIs.

